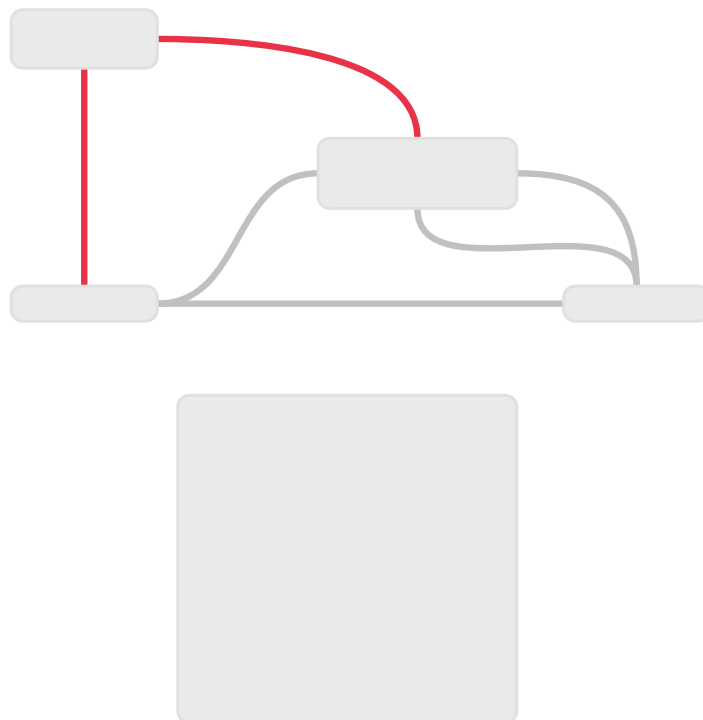


Cursul 10 - Fire de execuție în Java

Firele de execuție fac trecerea de la programarea secvențială la programarea concurentă. Pentru orice aplicație Java, la lansarea în execuție a acesteia, se crează automat și un fir de execuție numit *fir principal*. Într-un mecanism iterativ, acest fir poate crea alte fire de execuție care la rândul lor crează fire de execuție.

Stările unui fir de execuție

☐☐ Stările unui fir de execuție



Pentru un fir de execuție în Java, avem patru stări:

1. Ready
2. Running
3. Blocked
4. Dead

Lucrul cu fire de execuție în Java

Extinderea clasei Thread

```
public class Fir extends Thread {  
    public Fir(String nume) {
```

```

        super(ume)
    }
    public void run() {
        // cod aici care va fi executat de firul de execuție Fir
    }
}

```

Implementarea interfeței Runnable

```

public class SimpleThread implements Runnable {
    private Thread simpleThread = null;
    public SimpleThread() {
        if (simpleThread == null) {
            simpleThread = new Thread(this);
            simpleThread.start()
        }
    }
    public void run() {
        // cod aici care va fi executat de firul de execuție
        simpleThread
    }
}

```

OBS: Atunci când implementăm interfața `Runnable`, obligatoriu trebuie să definim metoda `run`.

Metode importante corespunzătoare clasei Thread

1. `public void start()` - are rolul de a porni firul de execuție într-o cale separată
2. `public void run()` - este o metodă pentru executarea efectivă a firului de execuție
3. `public final void setName(String name)` - are rolul de a defini numele firului de execuție. Avem și un getter aici.
4. `public final void setPriority(int priority)` - are rolul de a seta prioritatea firului de execuție în coada de execuție, din intervalul [1, 10]
5. `public final void setDaemon(boolean on)` - pentru a set un fir să ruleze în background
6. `public final void join(long millisec)` - metodă invocată când firul este în starea de blocare și se dorește trecerea la un fir secundar
7. `public void interrupt()` - are rolul de a întrerupe execuția firului
8. `public final boolean isAlive()` - are rolul de a returna `true` dacă firul există/nu se află în starea dead, false altfel.

Sincronizarea firelor de execuție

Sincronizarea firelor de execuție se bazează pe conceptul de *monitor*, care presupune "un lacăt" asociat unei resurse comune. Orice obiect care conține metode sincronizate are atașat și un monitor. Cuvântul cheie în Java pentru metode este `synchronized`